Enums

Lukáš Hozda

Patterns and error handling

NPRG082 · winter 2026/27

Compound data types in Rust

- In Rust, we have structs, enums, tuples and unions
- Enums are used in Rust extensively for things which have logical variants
- The variants can contain data, as we have seen
- To access data in enums, we need a core Rust concept: patterns & pattern matching

Enum example

```
enum Download {
    Receiving { bytes_received: u64 },
    Finished { file: String },
    Failed { error: String },
}
```

Variant construction

```
enum Download {
    Receiving { bytes_received: u64 },
    Finished(String),
    Cancelled,
}
fn main() {
    let state = Download::Receiving { bytes_received: 128 };
    if let Download::Receiving { bytes_received } = state {
        println!("receiving {bytes_received}");
    }
}
```

- **if-let** takes a **pattern**, not a condition
- if data matches the pattern, it is deconstructed and the if-let body is executed

Patterns

- Patterns describe the structure of values and can bind parts of them to variables
- We use them `match`, `if let`, `while let`, `let`, and function parameters
  - yes, **let** doesn't actually take an ident, it takes a pattern!
- Patterns can destructure tuples, structs, enums, arrays, or slices
- Patterns can be refutable (may fail to match) or irrefutable (always match)

These are particularly useful for working with enums.

Product and sum types

- A struct is a **product**, a choice for each field
- An enum is a **sum**, a choice of variant and its associated data
- `(bool, u8)`: $2 \times 256 = 512$ possible values
- `enum Level { Off, On(u8) }`: $1 + 256 = 257$ values

Recursive values

```
enum List {
    End,
    Node(i32, List),
}
fn main() {}
```

- Uh-oh
- E0072: the type would need infinitely large inline storage

Indirection with Box

```
enum List { End, Node(i32, Box<List>) }
fn sum(list: &List) -> i32 {
    match list {
        List::End => 0,
        List::Node(value, tail) => value + sum(tail),
    }
}
fn main() {
    let list = List::Node(7, Box::new(List::End));
    println!("{}", sum(&list));
}
```

Enum representation in memory

- Enums in Rust are implemented as tagged unions
- A variant's identity is its **discriminant**
- Optimizations can encode the discriminant in unused payload values, as we've seen last lecture with the Option niche, and this also happens with pointers

Niche optimization: Optional references

```
use std::mem::size_of;
fn main() {
    assert_eq!(size_of::<Option<&u8>>(), size_of::<&u8>());
    println!("true");
}
```

- A shared reference is non-null; null can encode `None`
- Equal size and alignment are guaranteed for this case

Match expressions

```
fn main() {
    let item = Some(7);
    let description = match item {
        Some(_) => "present",
        None => "absent",
    };
    println!("{description}");
}
```

- Arm results must have compatible types because match is an expression (like most things in Rust)
- _ is a wildcard pattern, it matches any value without binding it

Binding patterns

```
fn main() {
    let expected = 5;
    match 8 {
        expected => print!("{expected} "),
    }
    println!("{expected}");
}
```

- A binding pattern introduces a name into the scope of the match arm

Ranges and alternatives

```
fn main() {
    let code = 403;
    let message = match code {
        200..=299 => "success",
        401 | 403 => "denied",
        _ => "other",
    };
    println!("{message}");
}
```

- You can put the or | anywhere in a pattern

Match guards

```
fn main() {
    let expected = 5;
    match Some(5) {
        Some(value) if value == expected => println!("equal"),
        Some(_) => println!("different"),
        None => println!("absent"),
    }
}
```

- Handy way to avoid an extra nesting level and clarify things

Bindings with @

```
fn main() {
    match 204 {
        code @ 200..=299 => println!("{code}"),
        _ => println!("other"),
    }
}
```

- Bind the matched value and constrain it with another pattern

Struct and tuple patterns

```
struct Point { x: i32, y: i32 }
fn main() {
    let pair = (Point { x: 3, y: 4 }, 9);
    let (Point { x: horizontal, y }, _) = pair;
    println!("{horizontal} {y}");
}
```

- Field renaming, shorthand, and nested patterns

Slice patterns

```
fn ends(values: &[i32]) -> Option<(&i32, &i32)> {
    match values {
        [] | [_] => None,
        [first, .., last] => Some((first, last)),
    }
}
fn main() {
    println!("{:?}", ends(&[2, 4, 5]));
}
```

- `..` ignores the intervening elements, including zero elements

Bound subslices

```
fn main() {
    let values = [1, 2, 3, 4];
    if let [_, middle @ .., _] = values.as_slice() {
        println!("{middle:?}");
    }
}
```

- Here `middle` has type `&[i32]`

Early exits with let else

```
fn print_value(line: &str) {
    let Some((_, value)) = line.split_once('=') else {
        return;
    };
    println!("{value}");
}
fn main() { print_value("name=oak"); }
```

- A successful binding is available in the surrounding scope
- The `else` branch must diverge: `return`, `break`, `panic!`, …

Repeated matching with while-let

```
fn main() {
    let mut stack = vec![1, 2, 3];
    while let Some(value) = stack.pop() {
        print!("{value} ");
    }
}
```

- `pop` removes and returns the last element
- Loop until the refutable pattern fails, here on `None`

Moving inner data

```
fn main() {
    let name = Some(String::from("oak"));
    if let Some(text) = name {
        println!("{text}");
    }
    println!("{name:?}");
}
```

- `text` takes ownership of the inner `String`
- The original option may have been partially moved

Borrowing the inner data

```
fn main() {
    let name = Some(String::from("oak"));
    if let Some(text) = &name {
        print!("{text} ");
    }
    println!("{}", name.is_some());
}
```

- Matching through `&name` binds `text: &String`
- Reference binding is inferred from the borrowed value

Mutable pattern bindings

```
fn main() {
    let mut name = Some(String::from("oak"));
    if let Some(text) = &mut name {
        text.push('s');
    }
    println!("{}", name.unwrap());
}
```

- Here `text: &mut String`
- Mutation of inner data without moving it out

Explicit reference bindings

```
fn main() {
    let name = Some(String::from("oak"));
    if let Some(ref text) = name {
        print!("{text} ");
    }
    println!("{}", name.is_some());
}
```

- `ref text` borrows for the binding instead of moving
- `ref` creates a reference
- the opposite is an `&` pattern, which destructures one level of reference

Reference patterns

```
fn main() {
    let point = &(3, 4);
    let &(x, y) = point;
    println!("{x} {y}");
}
```

- The `&` pattern removes a reference layer during matching
- Here the bound coordinates are copied integers, not references

Absence and failure

- `Option<T>`: a value may be absent in normal operation
- `Result<T, E>`: an operation may fail with a described cause
- `Result<Option<T>, E>`: success can itself contain no value

Results

```
fn find_user(id: u64) -> Option<User>
fn parse_user(text: &str) -> Result<User, ParseError>
fn load_user(id: u64) -> Result<Option<User>, StorageError>
```

- The return type distinguishes these outcomes for the caller
- Error values can carry structured information
- `Err` contains just a value

Error propagation

```
use std::num::ParseIntError;
fn is_even(text: &str) -> Result<bool, ParseIntError> {
    let number = text.parse::<u32>()?;
    Ok(number % 2 == 0)
}
fn main() { println!("{:?}", is_even("42")); }
```

- The Try operator
- On `Ok`, unwrap the success value
- On `Err`, return early from the enclosing function

De-sugared propagation

```
let number = match text.parse::<u32>() {
    Ok(value) => value,
    Err(error) => return Err(error.into()),
};
```

- The enclosing function needs a compatible result type

Propagation on Option

```
fn initials(first: &str, last: &str) -> Option<(char, char)> {
    let first = first.chars().next()?;
    let last = last.chars().next()?;
    Some((first, last))
}
fn main() { println!("{:?}", initials("Ada", "Lovelace")); }
```

- With `Option`, `?` returns `None` early

Absence to error

```
#[derive(Debug)]
enum ConfigError { Missing }
fn main() {
    let setting: Option<u16> = None;
    let result = setting.ok_or(ConfigError::Missing);
    println!("{result:?}");
}
```

- `ok_or`: `Option<T>` to `Result<T, E>`
- `ok_or_else`: construct the error only for `None`

Application-specific error types

```
use std::num::ParseIntError;

#[derive(Debug)]
enum PortError {
    Parse(ParseIntError),
    Zero,
}
```

- Preserve the parsing error as internal

Error conversion and validation

```
use std::num::ParseIntError;
#[derive(Debug)]
enum PortError { Parse(ParseIntError), Zero }
fn parse_port(text: &str) -> Result<u16, PortError> {
    let port = text.parse::<u16>().map_err(PortError::Parse)?;
    if port == 0 { return Err(PortError::Zero); }
    Ok(port)
}
fn main() { println!("{:?}", parse_port("0")); }
```

- `map_err` transforms the error, leaving Ok(_) unchanged

Validation and types

```
use std::num::NonZeroU16;
fn main() {
    let rejected = NonZeroU16::new(0);
    let accepted = NonZeroU16::new(443);
    println!("{} {}", rejected.is_some(), accepted.is_some());
}
```

- After construction, a `NonZeroU16` always contains a nonzero value
- Return validated types when later code depends on that invariant

Error trait

- `std::error::Error`: a common trait that Err's inner data should implement
- Error variants identify cases callers can handle differently
- `Debug` (`{:?}`): programmer-facing/debug representation
- `Display` (`{}`): human-readable printing

Fallible main

```
use std::num::ParseIntError;
fn main() -> Result<(), ParseIntError> {
    let value = "42".parse::<u32>()?;
    println!("{value}");
    Ok(())
}
```

- `Ok(())` indicates successful completion
- Returning `Err` prints its `Debug` representation and exits unsuccessfully

Optional parsing / transpose

```
use std::num::ParseIntError;
fn setting(
    text: Option<&str>,
) -> Result<Option<u16>, ParseIntError> {
    text.map(|s| s.parse::<u16>()).transpose()
}
fn main() {
    println!("{:?} {:?}", setting(Some("42")), setting(None));
}
```

Result -> Option

```
fn main() {
    let malformed = "oak".parse::<u16>().ok();
    let out_of_range = "70000".parse::<u16>().ok();
    println!("{malformed:?} {out_of_range:?}");
}
```

- `ok()` discards the Err value
- `err()` would discard the Ok value